

Mainrun Assessment

Lam Le • Training a small GPT on Hacker News titles

1 Overview

The task was to lower validation loss on a 7 epoch budget for a small decoder-only transformer trained on Hacker News titles, starting from a baseline of 1.754. The provided `train.py` is a faithful GPT-2-small-style implementation, so most of the work involved replacing its 2019-era defaults with the choices that current small models and the modded-NanoGPT speedrun lineage have converged on, new architectures and modern standards, then re-tuning the hyperparameters after each change that altered training dynamics.

Metric	Baseline	Final	Change
Validation loss	1.754	0.881	-50%
Parameters	27,172,864	28,658,224	+5.5%
Seconds per epoch	FILL	2.15	FILL

Epoch count, random seed, dataset, validation fraction and the `evaluate()` function were left untouched throughout, as required.

2 Method

The working method was deliberately incremental:

1. **One change at a time.** Each architectural or optimiser change landed on its own commit and was evaluated against the previous best on the same setup, so every delta is attributable to a single cause.
2. **Re-sweep after behavioural changes.** Anything that alters training dynamics (optimiser, activation, normalisation, soft cap) invalidates the previous hyperparameter optimum. After each such change a Weights and Biases Bayesian sweep with Hyperband early termination re-tuned `lr`, `weight_decay`, `dropout` and, where relevant, the shape parameters `n_head`, `n_kv_heads`, `n_layer`, `d_model`, `qk_gain` and `final_logit_cap`. Sweep configs are in `sweep.yaml`, `sweep_confirm.yaml`, `lrwarmup.yaml` and `dropoutdecay.yaml`.
3. **Explore, then confirm.** Broad sweeps ran at a reduced budget to locate the promising region. A second sweep then ran at the true 7 epoch budget so the winning configuration transfers directly to the final evaluation rather than being tuned for a shorter horizon.
4. **Git as the experiment ledger.** Every change is a commit, so regressions are revertible and the loss history stays attributable.
5. **Sources, then verification.** Techniques were drawn from the NanoGPT and modded-NanoGPT speedruns, the parameter-golf results, and the design choices of current open models (Gemma, DeepSeek, GLM, Kimi, MiniMax, Qwen), then validated locally rather than assumed to transfer.

One piece of supporting infrastructure was added at the sweep boundary. A sweep can propose `n_head`, `n_kv_heads` and `d_model` independently, and most combinations violate the model's constraints. `sanitize_heads()` snaps any proposal to the nearest combination satisfying divisibility of `d_model` by

`n_head`, an even head dimension for RoPE, and divisibility of `n_head` by `n_kv_heads`, so no trial is lost to a crash.

3 Optimiser

3.1 SGD to AdamW to Muon

The baseline used plain SGD with cosine annealing. Swapping it for AdamW at the same configuration took validation loss to 1.545 on its own, the single largest change in the run. Muon was the intended target, but was not supported on the pinned PyTorch version, so AdamW served as the working optimiser while the rest of the model was modernised.

Muon (`modules/optim.py`, ported from Keller Jordan's reference implementation) runs SGD with momentum but orthogonalises the update matrix before applying it, replacing the momentum matrix with the orthogonal factor of its polar decomposition. The motivating observation is that updates to 2D transformer matrices are close to low rank, dominated by a handful of directions, so flattening the singular value spectrum amplifies the rare directions that carry little magnitude but real information. A full SVD per step is far too slow, so Muon approximates it with a five step quintic Newton-Schulz iteration in bf16 using only matrix multiplies. The coefficients (3.4445, -4.7750, 2.0315) are tuned to contract singular values toward 1 as quickly as possible rather than to converge exactly.

Orthogonalisation is only meaningful for hidden weight matrices, so the optimiser routes parameters by name: 2D non-embedding weights go to Muon, and everything else goes to an auxiliary Adam. That second group covers the embeddings, the tied head, the norms, and all of the scalars added later in this report (`lam`, `q_gain`, `unet_lam`). The rule is worth stating explicitly because it determines where any future parameter lands: a parameter goes to Muon if and only if it is 2D and its name contains neither `emb` nor `head`. This change helps the val loss significantly and makes the training process very stable and converge much faster compared to SGD

3.2 NorMuon

NorMuon closes the gap Muon leaves open. Muon fixes the condition number of the update but leaves its row norms highly non-uniform, so a few neurons still dominate each step. NorMuon keeps a second-moment EMA per row, normalises each row after orthogonalisation, and rescales so the RMS norm of the update matches what Adam would have produced. It combines matrix-level conditioning with neuron-level adaptivity, and costs one vector per matrix rather than a full second-moment tensor, so memory stays at roughly Muon's. During training, we record a significantly faster convergence in early steps and the overall training loss seems smoother.

3.3 Polar Express

Muon applies the same polynomial at every Newton-Schulz iteration, but the optimal polynomial when singular values are widely spread is not the optimal one when they are already clustered near 1. Classical Newton-Schulz is slowest in exactly the early iterations that matter most when only five are run. Polar Express instead solves a minimax problem at each step for the worst-case-optimal odd polynomial, giving both fast early contraction and fast asymptotic convergence, and it is bf16-stable. Landing it touches only `zeropower_via_newtonschulz5`: replace the single coefficient triple with a per-iteration list, and change the pre-scale denominator from $1e-7$ to $1e-2$. The measured gain at this scale was minimal, but it was kept because the cost was also minimal.

4 Data and tokenisation

4.1 CaseOps - Case operations

A byte-level BPE tokenizer treats “Rust” and “rust” as unrelated symbol sequences. The model therefore has to learn the semantics of each word twice, once per casing, which splits both the probability mass and the embedding capacity. Factoring casing out into a separate marker token lets the lowercase form carry all of the semantics while the marker carries the format signal alone. This is a standard trick in parameter-golf models.

The EDA in `eda/charset.ipynb` quantifies the payoff. Of the sampled titles, 9.74% are Title Case and 0.07% are ALL CAPS, so roughly one title in ten benefits directly. The effect on individual capitalised words is broader than the whole-title statistic suggests, since HN titles are dense with product names, acronyms and the “Show HN” prefix.

The markers do add tokens to the stream. Because the vocabulary is capped at 16,000, BPE reallocates the merges it would otherwise have spent on duplicate cased variants, so the net sequence-length cost is smaller than the raw marker count implies.

This change improved the loss significantly. Training loss perceived no negligible difference, but the `val_loss` was significantly better when applied CaseOps.

4.2 Sequence shape

The same EDA showed that the longest title in the dataset is 57 characters. Since titles are concatenated into one flat stream separated by `<eos>`, a context of 128 never contains more of the current document. It only contains more neighbouring and unrelated documents. Halving `block_size` to 64:

- still covers the longest title in a single window, so no in-document context is lost;
- removes half of the `pos_emb` parameters;
- makes attention four times cheaper per token pair, since cost is quadratic in sequence length.

The freed compute went into doubling `batch_size` to 128, which halves gradient variance per step. Development ran mostly on an M2 MacBook under MPS, where the reduced memory footprint also made the sweep loop practical.

5 Positional encoding

RoPE encodes position by rotating pairs of dimensions in the query and key vectors by an angle proportional to absolute position. For dimension pair i at position m the rotation angle is $m \cdot \theta_i$ with $\theta_i = 10000^{-2i/d}$. Because the attention score is an inner product, and rotating two vectors by angles m and n leaves their inner product dependent only on $m - n$, the attention logit becomes a function of relative position without any relative-position parameter table.

The alternative, a learned absolute position embedding, has to discover from data that position 40 and position 41 are adjacent, separately for every position. RoPE has that structure built in, adds no parameters, and reduces to a cached elementwise multiply. It is implemented in `_build_rope_cache` and `apply_rope`, applied to q and k after QK-norm and before the attention call, with $\theta = 10,000$.

6 Architecture

6.1 Bias removal

The MLP linears and the attention gate use `bias=False`. A bias immediately preceded by a normalisation layer is largely redundant, and every current frontier open model (Gemma, DeepSeek, Kimi, Qwen) as

well as modded-NanoGPT drops them. Removing them saved roughly 90k parameters and measurably reduced step time. Parameters that do nothing are strictly worse than no parameters: they cost memory traffic, optimiser state, and regularisation budget. As expected by the training and validation curve, we observed small improvement (largely due to small scale nature of the model) but a reduction in parameter count and training time per epoch.

6.2 QK-norm with a learnable per-head gain

QK-norm applies RMSNorm to each query and key head vector before the dot product. Without it nothing bounds the product of their norms, so attention logits can grow during training until the softmax saturates into a near one-hot distribution. A saturated softmax has vanishing gradients, and the model loses the ability to change what it attends to. QK-norm caps logit magnitude structurally and keeps attention distributions diverse. It is now standard in both modded-NanoGPT and NanoChat, both of which use RMSNorm rather than the L_2 norm of the original formulation.

QK-norm has a known cost: by bounding logits it also limits how sharply the model can focus on a genuinely important token. The fix used here is a learnable per-head scalar gain applied to the normalised query, which restores that freedom in a controlled way. Each head learns its own temperature, so heads doing broad averaging keep a low gain while heads doing precise lookup learn a high one. The gain initialises at 3.0, chosen by sweep. This pairing was one of the larger single wins in the run: it turns QK-norm from a stability mechanism with a quality cost into one with almost no cost.

6.3 XSA, exclusive self-attention

The standard attention output for token i is a convex combination of value vectors that includes the token's own v_i . Because a token's query usually resembles its own key, a substantial part of the output points along v_i , which spends attention capacity re-expressing information the token already has. The residual stream already preserves that content and the MLP already transforms it token-wise, so the component is redundant.

XSA removes it with a projection: subtract the component of the output aligned with the normalised own-value direction, leaving an output orthogonal to it. It adds no parameters, is fully parallel, and costs one normalisation plus one inner product and subtraction per head per position, which is linear in sequence length and negligible against the quadratic attention term.

There is a second and more interesting consequence. The attention softmax must sum to 1, so a token with no real contextual need is still forced to allocate weight somewhere, contaminating its output. The usual fix is an explicit sink token. Under XSA the model can simply attend heavily to itself and let the projection neutralise it, which gives the same escape valve for free.

This change is reflected on the `val_loss`, where we see the validation consistently lower loss throughout the whole curve.

6.4 Output gating

A sigmoid gate computed from the block input modulates the attention output elementwise before the output projection. This lets each channel learn how much attention output to admit as a function of the current token, a per-token per-channel soft switch. It is in the same family as the gating in Gemma-2 and the GLU variants, applied to the attention branch rather than to the MLP.

Since this is only a stability improvement, we recorded a slight improvement but insignificant at this scale. However, the number of added parameter was also insignificant so the change is deliberately kept.

6.5 Logit soft-capping

Taken from Gemma 2. A tanh squashes logits into the range $[-\text{cap}, +\text{cap}]$ smoothly and differentially. Near zero it is close to the identity, so it barely perturbs the well-calibrated regime and only bites where

the model is becoming extremely confident.

A 6 layer model on HN titles genuinely should not be certain about the next token, because titles are high entropy. Uncapped, the model can reduce training loss by driving a few easy predictions to near certainty, which produces very small gradients at those positions and effectively removes them from learning. Capping keeps the gradient signal alive across the whole batch. The cap is set to 15, matching modded-NanoGPT and NanoChat at this scale. The optimal value grows with model size, since larger models can justify more confidence, which is why Gemma uses a higher one.

6.6 ReLU squared instead of GELU

ReLU squared is $\max(0, x)^2$. Compared to GELU it is cheaper, a comparison and a multiply against an erf or tanh approximation, and it produces sparser activations, a hard zero below the threshold rather than GELU's small negative tail. SwiGLU is the more common choice in modern LLMs, but the evidence at small scale is mixed and NanoChat reports it losing to ReLU squared at several small scales. ReLU squared was chosen for its low cost and simplicity, and it is the usual choice among the parameter-golf models.

We recorded a good improvement in performance, and a smoother convergence in a training loss.

6.7 Grouped-query attention

GQA is implemented (`n_kv_head` with `repeat_interleave`) and was swept alongside the other shape parameters. Sharing key and value heads cut roughly 2M parameters and took epoch time from 10.4s to 7.6s on CPU, at the cost of slight late-run overfitting. The final configuration sets `n_kv_heads = n_head = 8`, so the deployed model runs full multi-head attention, with the GQA path left in place as a tuned-away option rather than a removed one so we can still have the option to scaledown the model while losing the least amount of capacity.

7 Training setup

7.1 Cautious weight decay

Decoupled weight decay shrinks every weight toward zero unconditionally. Cautious weight decay applies decay only where the update and the weight share a sign. If they differ, the gradient step is already moving that weight toward zero, so adding decay double-counts and over-shrinks. Gating removes that redundant pressure while keeping the regularisation where it does useful work. Both modded-NanoGPT and NanoChat use it, and modded-NanoGPT applies it to the Adam group as well, which is what `apply_weight_decay` does here.

7.2 Learning-rate warmup

The baseline began cosine annealing at the full learning rate on step 1. At the learning rates Muon prefers this produces a large, poorly conditioned first few steps. A short linear warmup, 5% of total steps, ramps from 1% of the target learning rate into the cosine schedule, implemented as a `SequentialLR` over `LinearLR` and `CosineAnnealingLR`. Warmup was swept jointly with learning rate and weight decay in `lrwarmup.yaml`.

The main observation on this was that it reduces both the train and val curves converges smoother. Through sweeping, we observed more configuration without a warmup period can cause overfitting. But when we have a warmup period, the model tend to settle down to the smaller adjustment period quicker, which leads it to work better with lower learning rate, and a more consistent convergence.

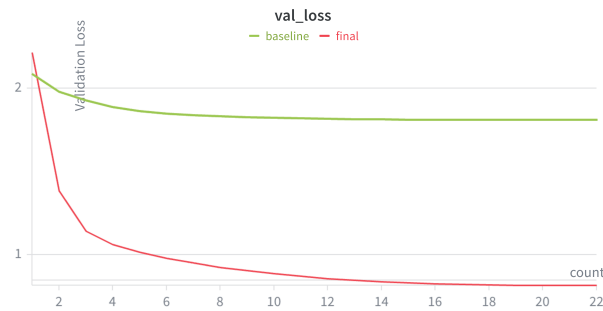


Figure 1: Baseline vs Final version Validation Loss

7.3 Final hyperparameters

Parameter	Baseline	Final	Set by
block_size	128	64	EDA on title length
batch_size	64	128	Compute freed by block_size
vocab_size	16,000	16,000	Unchanged
n_layer	6	6	Sweep
n_head	8	8	Sweep
n_kv_heads	n/a	8	Sweep
d_model	512	512	Sweep
dropout	0.1	0.09	Sweep
lr	6e-3	5e-3	Sweep
weight_decay	0.0	0.15	Sweep
warmup_frac	n/a	0.05	Sweep
qk_gain	n/a	3.0	Sweep
final_logit_cap	n/a	15.0	Literature, confirmed by sweep
rope_theta	n/a	10,000	Standard
optimiser	SGD	Muon + Adam	Section 3

The general shape of the result is that a small, low-capacity model on a short-context dataset prefers a high learning rate paired with strong decay: converge quickly, then regularise hard, because 7 epochs is not long enough to benefit from a cautious schedule.

8 Effect on the training curves

The changes fall into three groups by their signature on the curve.

- **Optimiser changes move the whole curve down and left.** The SGD to AdamW swap is visible within the first epoch and never gives the advantage back. Muon compounds this: the curve separates from AdamW early and the gap holds to the end of training.
- **Stability changes flatten the early spike.** QK-norm and warmup mainly change the first few hundred steps, removing the initial loss excursion that the uncapped, unwarmed baseline shows. Their effect on the final number is smaller than their effect on the shape.
- **Capacity and routing changes show up late.** Value residual and U-Net mixing barely separate in epoch 1 and open up over epochs 4 to 7, which is consistent with their mechanism: they matter once

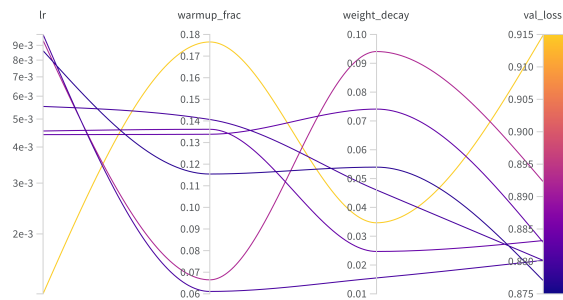


Figure 2: WandB sweeping to scan for better hyperparameter configuration.

the deep layers have drifted far enough from the shallow ones for the skip to carry new information.

Dropout and weight decay were tuned against the shape of the gap between training and validation loss in the final two epochs rather than against the final value alone, since the model reaches the overfitting regime inside the 7 epoch budget.

9 What did not work

- **Recurrence in attention.** No improvement at this depth and it cost step time.
- **Parallel attention and MLP branches** (the GPT-J and PaLM layout). Roughly neutral to slightly worse on loss. It is a throughput optimisation, and throughput was not the binding constraint here.
- **SwiGLU in place of ReLU squared.** Tested rather than assumed, and it did not win at this scale, matching the NanoChat finding.

10 References

1. Jordan, K. et al. Muon: an optimizer for hidden layers in neural networks. 2024.
2. Moonshot AI. Moonlight: scaling Muon. 2025.
3. Amsel, N. et al. The Polar Express: optimal matrix sign methods. 2025.
4. Zhou, Z. et al. Value Residual Learning (ResFormer). arXiv:2410.17897.
5. Google DeepMind. Gemma 2. arXiv:2408.00118. Logit soft-capping.
6. So, D. et al. Primer: searching for efficient transformers. arXiv:2109.08668. ReLU squared.
7. Chen, L. et al. Cautious Weight Decay. arXiv:2510.12402.
8. Su, J. et al. RoFormer: rotary position embedding. arXiv:2104.09864.
9. Karpathy, A. NanoChat, and Jordan, K. modded-nanogpt.
10. Conway, E. NanoGPT improvements, 2026. evanjayconway.com